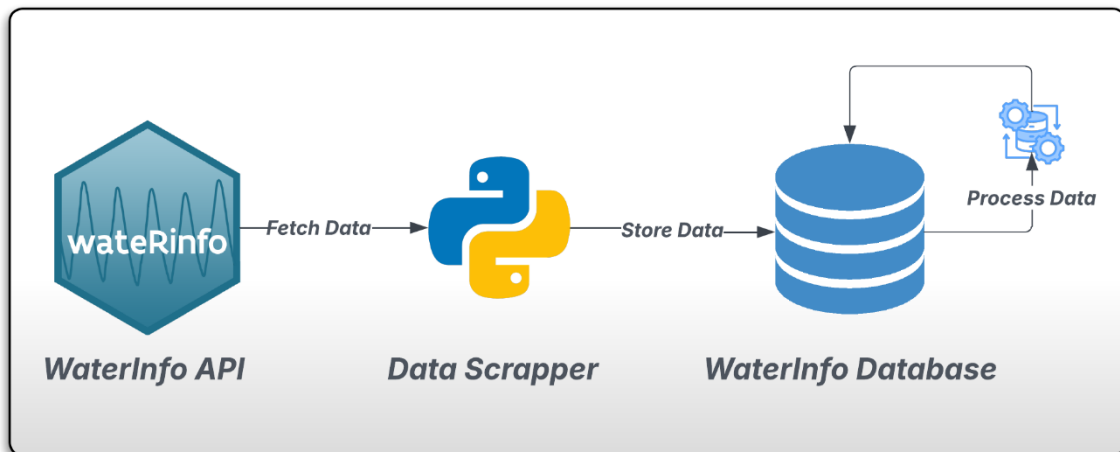


Waterinfo.be use case

Link: <https://github.com/ShehabEldeenAyman/WaterInfo>

This is the repository for waterinfo.be use case designed to transform sensory data into a linked data event stream. The stream should be web accessible and easy to fetch, store, and query.



What is Waterinfo Vlaanderen ?

- Waterinfo Vlaanderen is the official online portal of the Flemish government for real-time water-related data and forecasts.
- It is managed by the Vlaamse Milieumaatschappij (VMM).
- It provides comprehensive information on water levels, tides, rainfall, droughts, and flood risks throughout Flanders.

Key Features of Waterinfo Vlaanderen

- **Real-Time Water Levels and Flood Forecasts:** Access current water levels and 48-hour flood predictions for rivers and streams across Flanders.
- **Rainfall and Drought Monitoring:** Utilize detailed rainfall radar images and data from pluviometers to assess recent and predicted precipitation.
- **Tide Information:** Provides tidal forecasts and digital tide books.
- **Flood Risk Maps and Watercourse Data:** Offers access to flood risk maps.

Access to Data

- There are two libraries, both in Python and R, that facilitate fetching data from Waterinfo
- The libraries' development and maintenance are supported by Fluves. Part of the initial development of this package has been supported by VITO.

FLUVES



How does the data look?

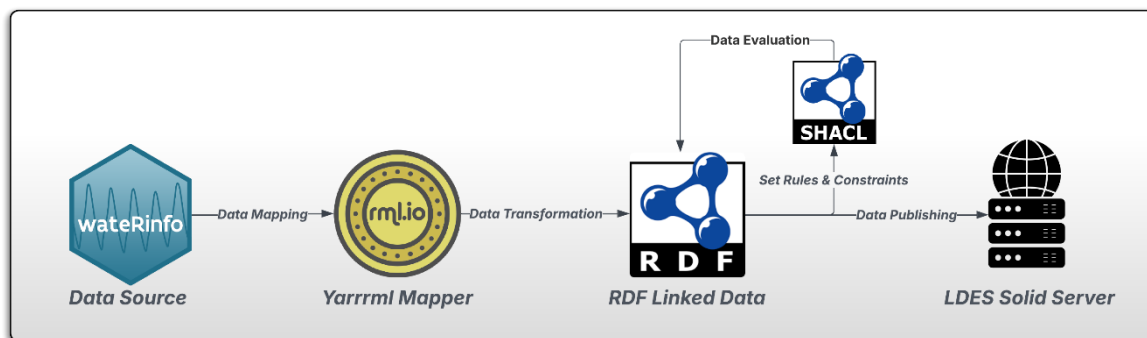
- Data is provided from two sources, which are VMM (Vlaamse Milieumaatschappij) and HIC (Hydrologisch Informatiecentrum).
- It contains information on both sensors and observations.
- For sensors, it contains information such as station location, address, etc.
- For observations, it contains information such as timestamp, reading value, unit measure, etc.

Challenges with the current data shape

- Readings are discrete and not linked together.
- Data does not conform to a standard model (e.g. SSN SOSA)
- Data is divided across several sources, which makes it difficult to query.
- Data is segmented properly, which makes it difficult to apply computations to.
- Data size quickly becomes an issue with fetching and computing.

Data pipeline

The proposed data pipeline is designed to transform data coming from Waterinfo.be into an LDES. Below is a breakdown of each component in the pipeline:



1. Data scraper

The first step is to fetch data from Waterinfo.be. The data is divided between VMM (Vlaamse Milieumaatschappij) and HIC (Hydrologisch Informatiecentrum) with separate APIs.

Fetching data from VMM or HIC will return a group list of stations, each with its own unique ID. Using those IDs, we can fetch information about the stations themselves along with information on the sensors. Each sensor has a unique Id and can be used to fetch historical data. Data is available up to 5 years but with usage limits.

The data scraper has been built as a standalone module that can be used separately. Please refer for Waterinfo.be for usage limits and API keys. The module can be found separately through this link: <https://github.com/ShehabEldeenAyman/waterinfo-scrap>

Group list attributes

- group_id

- group_name
- group_type

Station information attributes

- ts_id
- timestamp
- req_timestamp
- ts_value
- station_latitude
- station_longitude
- ts_name
- ts_shortcode
- station_no
- station_id
- station_name
- stationparameter_name
- stationparameter_no
- stationparameter_longname
- ts_unitname
- ts_unitsymbol
- parametertype_id
- parametertype_name
- ts_path
- dataprovider
- group_id

Sensor reading attributes

- Value
- InterpolationType
- QualityCode
- QualityCodeName
- QualityCodeDescription
- QualityCodeColor
- AggregationAccuracyPercentage
- AbsoluteValue AVInterpolationType
- AVQualityCode
- AVQualityCodeName
- AVQualityCodeDescription
- Runoff Value
- RV Interpolation Type
- RV Quality Code
- RV Quality Code Name
- RV Quality Code Description
- Stagesource Value
- ReturnPeriod
- Occurance Timestamp
- Occurance Count
- Timeseries Comment

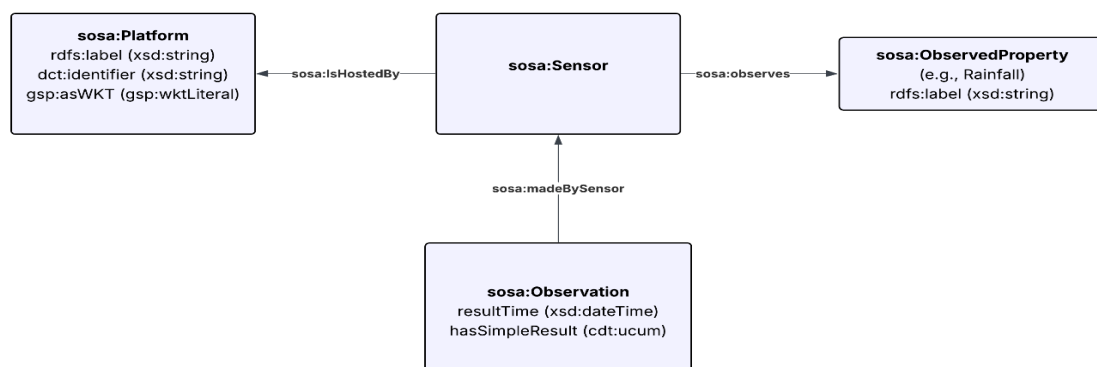
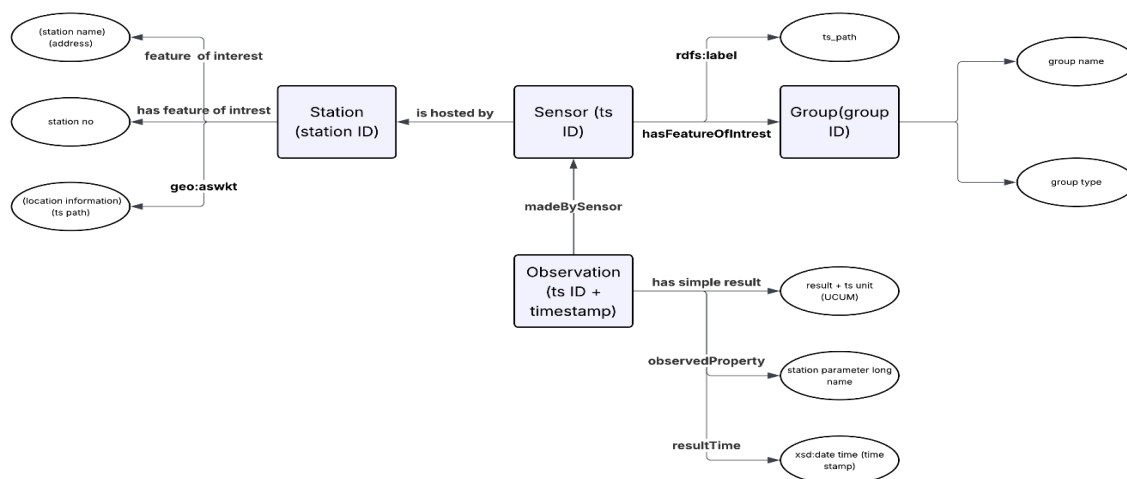
- Agent Comment
- Station Comment
- Parameter Comment
- Data Comment
- Release State
- ts_id

2. Data pre-processing

The data pre-processing module is designed to adjust attributes fetched from Waterinfo.be to be URI compliant. Names that include spaces or unsupported characters are modified. Also, unit measurements are changed to be UCUM “Unified Code for Units of Measure” compliant.

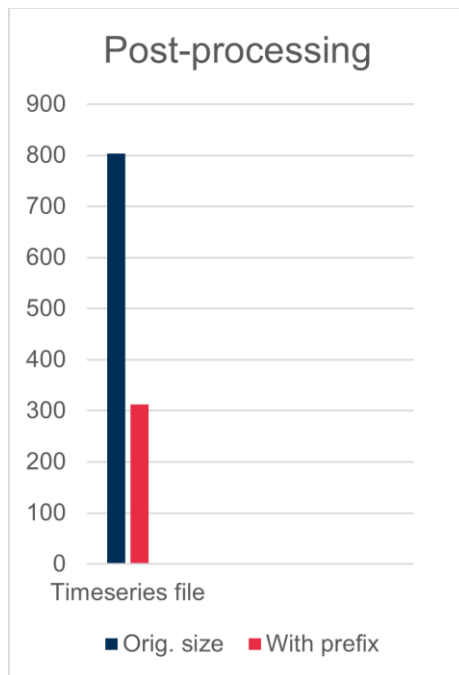
3. Mapping rules

Mapping rules are implemented to transform data from discrete CSV readings into a relational form. Data was mapped using SSN SOSA along with other standards in order to gain a more semantic understanding of the data. Also, to have a better linkage between data entities. Mapping was achieved by using RML Mapper and RML Streamer modules to transform CSV to RDF.



4. Data post-processing

Data post-processing module is designed to be a generic module that modifies RDF files to add back prefixes, resulting in a huge reduction in file size which in return reduces the load on computational and storage resources.



With a timeseries file of 591349 records, the prefix post-processing technique was able to reduce the file size to 39% of its original size.

The data post-processing module was designed to be generic, and it can work on any RDF-compliant data. The module has been published separately and can be found on: <https://github.com/ShehabEldeenAyman/rdf-prefix-binder>

5. Rules and Constraints

Rules and constraints are designed to make sure that the data added to the pre-determined model is compliant. Rules are written in the form of SHACL shapes and are validated against data entries. Rules are flexible in the sense that any rule that needs to be enforced on the data can be specified.

6. LDES conversion (In development)

The LDES conversion module is in development as a standalone module. Its goal is to convert RDF-compliant data to LDES along with adding TREE hypermedia. Currently, the module is in early development with focus on sensory observations coming from waterinfo.be. Later, it will take a more general form to be compatible with all kinds of star pattern observations.

```

10 ex:SaltSensor1 a sosa:Sensor ;
11     rdfs:label "Salt Concentration Sensor" ;
12     sosa:observes ex:SaltConcentration .
13
14 ex:WaterSampleLocation1 a sosa:FeatureOfInterest ;
15     rdfs:label "Water Sample Location 1" .
16
17 # Observation 1
18 ex:Obs1 a sosa:Observation ;
19     sosa:madeBySensor ex:SaltSensor1 ;
20     sosa:hasFeatureOfInterest ex:WaterSampleLocation1 ;
21     sosa:observedProperty ex:SaltConcentration ;
22     sosa:resultTime "2025-05-15T08:00:00Z"^^xsd:dateTime ;
23     sosa:hasSimpleResult "3500"^^xsd:double ;
24     qudt:unit unit:PPM .

```

RDF-compliant data

```

18 # 2. Define the event stream
19
20 <#EventStream> a ldes:EventStream ;
21     rdfs:label "Salt Concentration LDES Stream" ;
22     ldes:timestampPath sosa:resultTime ;
23     # order events by the sosa:resultTime property:
24     tree:member ex:Obs1, ex:Obs2, ex:Obs3 . #only include members of the first page
25
26
27 # 5. The actual observations (same as before)
28 ex:Obs1 a sosa:Observation ;
29     sosa:madeBySensor ex:SaltSensor1 ;
30     sosa:hasFeatureOfInterest ex:WaterSampleLocation1 ;
31     sosa:observedProperty ex:SaltConcentration ;
32     sosa:resultTime "2025-05-15T08:00:00Z"^^xsd:dateTime ;
33     sosa:hasSimpleResult "3500"^^xsd:double ;
34     qudt:unit unit:PPM .

```

LDES data